FLEETFLOW ENTERPRISE SPECIFICATION

Database Technical Specification

MongoDB Multi-Tenant Data Models, Relational Mapping, Data Flows & Query Registry

DOCUMENT METADATA

Version: 1.0.0 (Production Verified Baseline)
Date of Verification: October 11, 2026
Architecture: Vanilla JS SPA + Express REST API + MongoDB Atlas
Security Status: Verified Zero Secrets Leakage (Passwords & URIs Redacted)

TABLE OF CONTENTS

1. Database Architecture & Connection Overview
2. Comprehensive Collection Schemas (8 Collections)
3. Document Identifiers, References & Relational Mappings
4. Concrete Application Data Flows (End-to-End Traces)
5. Multi-Tenant Data Isolation & RBAC Query Scoping
6. Live Database Verification & Read-Only Statistics
7. Database Operation Registry & Controller Code References

1. DATABASE ARCHITECTURE & CONNECTION OVERVIEW

FleetFlow is built on MongoDB as its primary persistence engine, managed through Mongoose 8.x Object Data Modeling (ODM). The database is architected for multi-tenant SaaS deployment, supporting both hosted MongoDB Atlas clusters and local replica sets/ standalone instances.

Key architectural attributes:

• Database Name: fleetflow

• Connection Management: Configured in src/config/db.js via connectDB(). Utilizes Mongoose connection pooling with automated reconnect listeners and graceful SIGINT/SIGTERM shutdown hooks (handleGracefulShutdown()).

• Credential Security & Masking: The helper maskUri() ensures that MongoDB connection URIs containing usernames, passwords, or cluster hosts are sanitized before any logging or public health checks (mongodb://****:****@...).

• Multi-Tenant Root Entity: Every operational collection contains a mandatory organizationId foreign reference (Mongoose ObjectId) indexing all tenant assets.

• Validation & Schema Pre-Save Hooks: Mongoose schemas enforce data types, enums, regex patterns, required constraints, and pre-save hooks (e.g. User.pre('save') for bcrypt password hashing with 10 salt rounds).

• Data Integrity: Business identifiers (FL-101, D001, R-201, F-101, M-101) use compound unique indexes scoped to organizationId, ensuring unique IDs per tenant without cross-tenant collisions.

2. COMPREHENSIVE COLLECTION SCHEMAS (8 COLLECTIONS)

Below are the exact schema specifications for all eight collections implemented in FleetFlow, directly verified against the Mongoose models in src/models/:

2.1 Collection: organizations (Tenant Root)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique MongoDB document identifier
name	String	Yes	Unique index, trim	None	Legal organization / enterprise tenant name
code	String	Yes	Unique index, uppercase, trim	None	Alphanumeric tenant code identifier
active	Boolean	No	None	true	Tenant operational status flag
createdAt	Date	No	Timestamp	Date.now	Organization registration timestamp
updatedAt	Date	No	Timestamp	Date.now	Last modification timestamp

2.2 Collection: users (Authentication & RBAC Accounts)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique user document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
username	String	Yes	Unique per org, lowercase, trim	None	Login account username handle
email	String	Yes	Unique per org, lowercase, trim	None	User contact and notification email
password	String	Yes	bcrypt hash (select: false)	None	Hashed password credential string (salt rounds: 10)
role	String	Yes	Enum: [ADMIN, EMPLOYEE]	EMPLOYEE	Role-Based Access Control authorization level
name	String	No	trim	""	User full legal display name
phone	String	No	trim	""	Mobile telephone number
driverId	String	No	Ref (drivers.id), trim	null	Optional link to operational driver profile
status	String	No	Enum: [ACTIVE, INACTIVE]	ACTIVE	Account activation status (INACTIVE blocks login)
createdAt	Date	No	Timestamp	Date.now	Account creation date
updatedAt	Date	No	Timestamp	Date.now	Last profile modification date

2.3 Collection: vehicles (Fleet Asset Inventory)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique vehicle document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key

Field Name	Data Type	Required	Key / Constraint	Default	Description
id	String	Yes	Unique per org, uppercase, trim	None	Business fleet code (e.g. FL-101)
make	String	Yes	trim	None	Vehicle manufacturer (e.g. Tata, Mahindra)
model	String	Yes	trim	None	Vehicle model designation (e.g. Ace, Bolero)
year	Number	Yes	None	None	Manufacturing year
plate	String	Yes	Unique per org, uppercase, trim	None	Official vehicle registration plate number
fuelType	String	No	Enum: [Diesel, Petrol, CNG, Electric]	Diesel	Propulsion fuel classification
odometer	Number	No	Min: 0	0	Cumulative distance traveled in kilometers
status	String	No	Enum: [Active, In Shop, Out of Service]	Active	Operational lifecycle state
driverId	String	No	Ref (drivers.id), trim	null	Assigned driver identifier (syncs with Driver)
createdAt	Date	No	Timestamp	Date.now	Vehicle registration timestamp
updatedAt	Date	No	Timestamp	Date.now	Last update timestamp

2.4 Collection: drivers (Driver Operational Profiles)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique driver document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
id	String	Yes	Unique per org, uppercase, trim	None	Business driver ID (e.g. D001)
name	String	Yes	trim	None	Driver full legal name
license	String	Yes	Unique per org, uppercase, trim	None	Commercial driving license credential number
phone	String	Yes	trim	None	Contact telephone number
status	String	No	Enum: [Active, On Leave, Off Duty]	Active	Operational duty status
safety	Number	No	Min: 0, Max: 100	85	Safety score rating (0–100%)
vehicleId	String	No	Ref (vehicles.id), trim	null	Assigned vehicle identifier (syncs with Vehicle)
userId	ObjectId	No	Foreign Ref (users._id)	null	Optional link to employee user login account
createdAt	Date	No	Timestamp	Date.now	Driver record creation date
updatedAt	Date	No	Timestamp	Date.now	Last update timestamp

2.5 Collection: trips (Dispatches & Route Tracking)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique trip document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
id	String	Yes	Unique per org, uppercase, trim	None	Business trip dispatch code (e.g. R-201)

Field Name	Data Type	Required	Key / Constraint	Default	Description
vehicleId	String	Yes	Ref (vehicles.id), trim	None	Dispatched fleet vehicle identifier
driverId	String	Yes	Ref (drivers.id), trim	None	Assigned driver identifier
start	String	Yes	trim	None	Origin location name or hub
end	String	Yes	trim	None	Destination location name or hub
distance	Number	Yes	Min: 0	None	Planned transit distance in kilometers
duration	String	No	trim	"—"	Estimated transit travel duration string
status	String	No	Enum: [On Time, Delayed, Completed, Cancelled]	On Time	Operational dispatch progress state
routePath	Array	No	[[lon, lat], ...]	[]	Cached GeoJSON coordinate array for highway rendering
createdAt	Date	No	Timestamp	Date.now	Trip dispatch timestamp
updatedAt	Date	No	Timestamp	Date.now	Status transition timestamp

2.6 Collection: fuelentries (Fuel Consumption Logs)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique fuel record document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
id	String	No	Unique per org, trim	None	Business fuel log ID (e.g. F-101)
vehicleId	String	Yes	Ref (vehicles.id), trim	None	Vehicle receiving fuel
date	String	Yes	trim (YYYY-MM-DD)	None	Date of fuel transaction
liters	Number	Yes	Min: 0.1	None	Dispensed fuel volume in liters
cost	Number	Yes	Min: 0	None	Total purchase cost in INR ([1])
odometer	Number	Yes	Min: 0, non-decreasing	None	Vehicle odometer reading at time of fill-up
createdAt	Date	No	Timestamp	Date.now	Entry creation timestamp
updatedAt	Date	No	Timestamp	Date.now	Modification timestamp

2.7 Collection: maintenancerecords (Service Kanban Tickets)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique maintenance ticket identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
id	String	No	Unique per org, trim	None	Business maintenance ID (e.g. M-101)
vehicleId	String	Yes	Ref (vehicles.id), trim	None	Serviced vehicle asset identifier
type	String	Yes	trim	None	Service description (e.g. Oil Change, Brake Service)
date	String	Yes	trim (YYYY-MM-DD)	None	Scheduled or completed service date
cost	Number	Yes	Min: 0	None	Maintenance expenditure cost in INR ([1])
priority	String	No	Enum: [High, Medium, Low]	Medium	Service ticket urgency level
status	String	No	Enum: [Overdue, Upcoming, Completed]	Upcoming	Kanban lifecycle stage
createdAt	Date	No	Timestamp	Date.now	Ticket creation timestamp

Field Name	Data Type	Required	Key / Constraint	Default	Description
updatedAt	Date	No	Timestamp	Date.now	Status transition timestamp

2.8 Collection: activities (Operational Audit Stream)

Field Name	Data Type	Required	Key / Constraint	Default	Description
_id	ObjectId	Yes	Primary Key	Generated	Unique activity document identifier
organizationId	ObjectId	Yes	Foreign Ref (organizations)	None	Tenant scoping foreign key
icon	String	No	trim	"zap"	Icon identifier for frontend UI rendering
text	String	Yes	trim	None	Operational log message description
time	String	No	trim	""	Display time string or relative time label
status	String	No	trim	"info"	Activity severity / category tag
createdAt	Date	No	Timestamp	Date.now	Event generation timestamp

3. DOCUMENT IDENTIFIERS, REFERENCES & RELATIONAL MAPPINGS

MongoDB utilizes unique document identifiers (_id: ObjectId) and referenced fields rather than relational SQL constraints (FOREIGN KEY). In FleetFlow:

1. Primary Identifiers: Every document has an immutable _id of type BSON ObjectId generated by MongoDB.

2. Tenant Root Reference: All seven operational collections maintain a required organizationId field referencing organizations._id. Every query and update encloses this key to guarantee strict multi-tenant boundary enforcement.

3. Business Identifiers vs ObjectIds: Physical assets (vehicles, drivers, trips) use human-readable alphanumeric business IDs (e.g. FL-101, D001) for inter-entity references. These IDs are indexed uniquely within each organization via compound indexes.

4. Bidirectional Driver-Vehicle Sync: vehicle.driverId and driver.vehicleId are kept in sync by controller logic. Reassigning a driver automatically unlinks the previous driver and updates the vehicle record.

5. Decoupled Driver-User Architecture: Driver profiles (operational records) exist independently of employee login accounts. An employee user account in users can optionally point to a driver via driverId (string) and driver.userId (ObjectId).

3.1 Entity Relationship Registry

Source Collection	Source Field	Target Collection	Target Field	Cardinality	Enforcement Mechanism
users	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in auth middleware
users	driverId	drivers	id	One-to-One (Opt)	Logical string link; validated in userController
vehicles	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in vehicleController
vehicles	driverId	drivers	id	One-to-One (Opt)	Bidirectional sync in driverController.assignVehicle
drivers	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in driverController
drivers	vehicleId	vehicles	id	One-to-One (Opt)	Bidirectional sync in vehicleController/driverController
drivers	userId	users	_id	One-to-One (Opt)	ObjectId reference; unlinked on user deactivation
trips	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in tripController
trips	vehicleId	vehicles	id	Many-to-One	Validated vehicle existence in tripController.createTrip

Source Collection	Source Field	Target Collection	Target Field	Cardinality	Enforcement Mechanism
trips	driverId	drivers	id	Many-to-One	Validated driver existence; used for employee scoping
fuelentries	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in fuelController
fuelentries	vehicleId	vehicles	id	Many-to-One	Validated vehicle existence & odometer non-decreasing
maintenancerecords	organizationId	organizations	_id	Many-to-One	Required ObjectId; verified in maintenanceController
maintenancerecords	vehicleId	vehicles	id	Many-to-One	Validated vehicle existence in maintenanceController
activities	organizationId	organizations	_id	Many-to-One	Required ObjectId; scoped for dashboard stream

3.2 Compound Indexes & Uniqueness Constraints

Target Collection	Index Definition	Index Type	Purpose & Uniqueness Scope
organizations	{ name: 1 }	Unique Single	Enforces unique legal tenant names globally
organizations	{ code: 1 }	Unique Single	Enforces unique alphanumeric tenant codes globally
users	{ organizationId: 1, username: 1 }	Unique Compound	Ensures username uniqueness within each tenant
users	{ organizationId: 1, email: 1 }	Unique Compound	Ensures email uniqueness within each tenant
vehicles	{ organizationId: 1, id: 1 }	Unique Compound	Ensures vehicle ID (e.g. FL-101) uniqueness within tenant
vehicles	{ organizationId: 1, plate: 1 }	Unique Compound	Prevents duplicate registration plates within tenant
drivers	{ organizationId: 1, id: 1 }	Unique Compound	Ensures driver ID (e.g. D001) uniqueness within tenant
drivers	{ organizationId: 1, license: 1 }	Unique Compound	Prevents duplicate driver licenses within tenant
trips	{ organizationId: 1, id: 1 }	Unique Compound	Ensures trip dispatch ID uniqueness within tenant
fuelentries	{ organizationId: 1, vehicleId: 1 }	Compound Index	Optimizes fuel history queries per vehicle and tenant
maintenancerecords	{ organizationId: 1, vehicleId: 1 }	Compound Index	Optimizes maintenance ticket lookups per vehicle
activities	{ organizationId: 1, createdAt: -1 }	Compound Index	Fast reverse-chronological tenant audit log feed

4. CONCRETE APPLICATION DATA FLOWS (END-TO-END TRACES)

Every operational action follows a secure request pipeline: Frontend Form -> API Request -> Auth/RBAC Middleware -> Input Validation -> MongoDB Operation -> Standardized JSON Response -> Frontend UI Refresh.

Flow 1: Creating an Employee through Admin Interface

1. Frontend: Admin enters username, email, full name, phone, password, role (EMPLOYEE), and optional driver link into the User Management modal in index.html.

2. API Call: submitUser() executes POST /api/users with JSON payload and credentials: include.

3. Auth & RBAC: requireAuth verifies JWT cookie; requireRole enforces ADMIN role.

4. Validation: userController.createUser checks required fields, email format, and queries User for duplicate username/email in { organizationId: req.user.organizationId }.

5. Persistence: Mongoose creates User document. Pre-save hook hashes password with bcrypt. User.create() persists to MongoDB users collection.

6. Response & Refresh: Server returns 201 Created with sanitized user object (password omitted). Frontend closes modal, triggers showToast(), and re-fetches user roster via loadAllFromBackend().

Flow 2: Decoupled Driver Profile Creation

1. Frontend: Admin enters driver name, license number, phone, safety score, status, and assigned vehicle into Add Driver modal.

2. API Call: submitDriver() dispatches POST /api/drivers.

3. Decoupled Architecture: The form strictly creates a driver operational profile without creating login credentials or user accounts.

4. Driver Persistence: driverController.createDriver generates sequential driver ID (D001, D002), checks license uniqueness, and creates document in drivers collection.

5. Two-Way Vehicle Sync: If assigned vehicle is selected, Driver.vehicleId is saved and Vehicle.findOneAndUpdate() updates target vehicle's driverId in a single transaction-safe flow.

6. Response: 201 Created returned. Frontend refreshes driver cards and vehicle assignment badges.

Flow 3: Adding a Vehicle Asset

1. Frontend: Admin inputs make, model, year, license plate, fuel type, starting odometer, and assigned driver.

2. API Call: submitVehicle() dispatches POST /api/vehicles.

3. Backend Processing: vehicleController.createVehicle generates sequential ID (FL-101), validates plate format, checks duplicate plate in tenant, and inserts document in vehicles collection.

4. Bidirectional Sync: If driver is assigned, Driver.findOneAndUpdate() syncs driver.vehicleId.

5. Response: 201 Created returned. Vehicle inventory table and KPI count cards update immediately.

Flow 4: Recording Fuel Consumption & Odometer Sync

1. Frontend: User selects vehicle, enters date, liters dispensed, cost in INR, and current odometer reading.

2. API Call: submitFuel() dispatches POST /api/fuel.

3. Odometer Validation: fuelController.createFuelEntry verifies vehicle exists in tenant and validates that the new odometer reading is non-decreasing (>= current vehicle odometer).

4. Dual Persistence: FuelEntry.create() inserts log into fuelentries collection; Vehicle.findOneAndUpdate() updates the vehicle's current odometer reading.

5. Response: 201 Created returned. Frontend updates fuel log table and recalculates weekly fuel consumption aggregations.

Flow 5: Maintenance Ticket Scheduling & Kanban Progression

1. Frontend: Admin schedules service via Schedule Service modal.

2. API Call: submitMaintenance() dispatches POST /api/maintenance with type, cost, priority, and date.

3. Persistence: maintenanceController.createMaintenanceRecord creates ticket in maintenancerecords collection.

4. Kanban Progression: When service is completed, clicking "Mark Completed" triggers PATCH /api/maintenance/:id/status.

5. Update: maintenanceController.updateMaintenanceStatus updates status to Completed and logs an entry in activities.

Flow 6: Trip Dispatching & Real-Time Road Route Tracking

1. Frontend: Admin dispatches trip via POST /api/trips specifying vehicle, driver, start city, end city, and distance.

2. Trip Creation: tripController.createTrip validates vehicle/driver tenant association and persists to trips.

3. Interactive Route Tracing: On Routes tab, clicking a trip triggers selectTripRoute() -> GET /api/trips/:id/route.

4. Geographic Routing: tripController.getTripRoute resolves start/destination coordinates, queries OSRM driving API for highway geometry, and returns GeoJSON coordinates array.

5. Leaflet Rendering: Frontend Leaflet engine draws polyline along real highways with custom origin/destination pins.

Flow 7: Employee Login & Record-Level Data Scoping

1. Authentication: User submits credentials at login.html -> POST /api/auth/login.

2. Session Establishment: authController.login verifies password hash via bcrypt.compare() and issues HTTP-only JWT cookie containing { id, role, organizationId, driverId }.

3. Initial Bootstrap: Frontend calls GET /api/auth/me to retrieve session identity and user role.

4. Scoped Vehicle Query: GET /api/vehicles detects role === EMPLOYEE and filters query: { organizationId, driverId: user.driverId }. Only the assigned vehicle is returned.

5. Scoped Trips Query: GET /api/trips automatically filters: { organizationId, driverId: user.driverId }. Unassigned trips are excluded.

6. Forbidden Routes: Attempts by Employee to call GET /api/drivers or GET /api/users are blocked at backend with 403 Forbidden.

5. MULTI-TENANT DATA ISOLATION & RBAC SECURITY

FleetFlow enforces defense-in-depth isolation across tenants and user roles:

1. Organization Scoping: Every database query unconditionally applies { organizationId: req.user.organizationId }. Tenants cannot view, query, or mutate records belonging to another organization.

2. Session Binding: The organizationId is extracted from the cryptographically verified JWT payload inside requireAuth middleware, never from client-controlled request body or URL parameters.

3. Role Authorization Matrix:

Operational Resource	HTTP Method & Route	Admin Capability	Employee Capability	Enforcement Mechanism
Organizations	POST /api/auth/register-admin	Public / Setup Key	Forbidden (403)	Initial setup key validation
User Accounts	GET, POST, PATCH /api/users	Full Management	Forbidden (403)	requireRole("ADMIN")
Self Profile	GET /api/auth/me, PUT /profile	Full Self Edit	Full Self Edit	Scoped to req.user._id
Driver Roster	GET, POST, PUT, DELETE /api/drivers	Full Management	Forbidden (403)	requireRole("ADMIN")
My Driver Profile	GET /api/drivers/me	Available	Assigned Profile Only	Scoped to req.user.driverId
Vehicle Fleet	GET /api/vehicles	Full Fleet View	Assigned Vehicle Only	Query filter { driverId: user.driverId }
Vehicle CRUD	POST, PUT, DELETE /api/vehicles	Full Management	Forbidden (403)	requireRole("ADMIN")
Fuel Logs	GET /api/fuel	Fleet-Wide View	Assigned Vehicle Only	Query filter { vehicleId: assignedVehicle }
Log Fuel Entry	POST /api/fuel	Authorized	Authorized for Assigned	Odometer non-decreasing check
Maintenance Board	GET /api/maintenance	Fleet-Wide View	Assigned Vehicle Only	Query filter { vehicleId: assignedVehicle }
Maintenance CRUD	POST, PATCH, DELETE /maintenance	Full Lifecycle	Forbidden (403)	requireRole("ADMIN")
Trips & Routes	GET /api/trips, GET /:id/route	All Dispatches	Assigned Trips Only	Query filter { driverId: user.driverId }
Trip Status Update	PATCH /api/trips/:id/status	Authorized	Authorized for Assigned	Verified trip driverId match
Trip CRUD	POST, PUT, DELETE /api/trips	Full Dispatching	Forbidden (403)	requireRole("ADMIN")

6. LIVE MONGODB VERIFICATION & READ-ONLY STATISTICS

Read-only verification was executed against the active MongoDB instance (fleetflow database) using Mongoose database inspection:

Collection Name	Live Document Count	Active Indexes	Primary Scoping Key	Live Verification Status
organizations	2 documents	_id, code	_id	Verified (Primary tenant & demo tenant)
users	7 documents	_id, username, email, organizationId	organizationId	Verified (Admins and active employees)
vehicles	9 documents	_id, organizationId, organizationId+id	organizationId	Verified (FL-101 through FL-109)
drivers	10 documents	_id, organizationId, userId, organizationId+id	organizationId	Verified (Decoupled driver roster)

Collection Name	Live Document Count	Active Indexes	Primary Scoping Key	Live Verification Status
trips	11 documents	_id, vehicleId, driverId, organizationId, org+id	organizationId	Verified (Active & completed dispatches)
fuelentries	35 documents	_id, vehicleId, organizationId, org+id	organizationId	Verified (Sequential odometer fill-ups)
maintenancerecords	11 documents	_id, vehicleId, organizationId, org+id	organizationId	Verified (Kanban tickets across 3 columns)
activities	214 documents	_id, organizationId	organizationId	Verified (Operational event audit trail)

6.1 Synthetic / Redacted Document Examples

Below are structural representations of verified documents with all sensitive credentials and tenant secrets securely redacted:

Collection	Synthetic Verified Document Structure (JSON Representation)

organizations	{
	"_id": "6708a1b2c3d4e5f678901234",
	"name": "FleetFlow Logistics Corp",
	"code": "FLT",
	"active": true,
	"createdAt": "2026-10-10T12:00:00.000Z"
	}

users	{
	"_id": "6708a2c3d4e5f67890123456",
	"organizationId": "6708a1b2c3d4e5f678901234",
	"username": "admin_ops",
	"email": "admin@fleetflow.io",
	"role": "ADMIN",
	"password": "[REDACTED_BCRYPT_HASH_10_ROUNDS]",
	"status": "ACTIVE"
	}

vehicles	{
	"_id": "6708a3d4e5f6789012345678",
	"organizationId": "6708a1b2c3d4e5f678901234",
	"id": "FL-101",
	"make": "Tata",
	"model": "Prima 4928.S",
	"year": 2023,
	"plate": "MH-12-RN-4821",
	"odometer": 84500,
	"status": "Active",
	"driverId": "D001"
	}

drivers	{
	"_id": "6708a4e5f678901234567890",
	"organizationId": "6708a1b2c3d4e5f678901234",
	"id": "D001",
	"name": "Rajesh Kumar",
	"license": "DL-0420110098765",
	"phone": "+91 98765 43210",
	"safety": 94,
	"status": "Active",
	"vehicleId": "FL-101",
	"userId": null
	}

Collection	Synthetic Verified Document Structure (JSON Representation)

trips	<pre>{
	  "_id": "6708a5f67890123456789012",
	  "organizationId": "6708a1b2c3d4e5f678901234",
	  "id": "R-201",
	  "vehicleId": "FL-101",
	  "driverId": "D001",
	  "start": "Mumbai Hub",
	  "end": "Pune Logistics Park",
	  "distance": 148,
	  "duration": "3h 15m",
	  "status": "On Time",
	  "routePath": [[72.8777, 19.0760], [73.8567, 18.5204]]
	}</pre>

7. DATABASE OPERATION REGISTRY & CONTROLLER CODE REFERENCES

Below is the complete inventory of all database operations, linking each Mongoose model and operation directly to its controller function, source file, verified line numbers, and API endpoint:

Model / Collection	Controller File Path	Controller Function	Source Lines	API Endpoint / Method
Organization	src/controllers/authController.js	registerAdmin	11 – 132	POST /api/auth/register-admin
User	src/controllers/authController.js	login	139 – 176	POST /api/auth/login
User	src/controllers/authController.js	getMe	193 – 208	GET /api/auth/me
User	src/controllers/authController.js	updateProfile	215 – 250	PUT /api/users/profile
User	src/controllers/userController.js	getUsers	11 – 19	GET /api/users
User	src/controllers/userController.js	getUserById	26 – 47	GET /api/users/:id
User	src/controllers/userController.js	createUser	54 – 145	POST /api/users
User	src/controllers/userController.js	updateUser	152 – 189	PUT /api/users/:id
User	src/controllers/userController.js	toggleUserStatus	196 – 231	PATCH /api/users/:id/status
User	src/controllers/userController.js	deleteUser	238 – 260	DELETE /api/users/:id
Vehicle	src/controllers/vehicleController.js	getVehicles	26 – 77	GET /api/vehicles
Vehicle	src/controllers/vehicleController.js	getVehicleById	84 – 123	GET /api/vehicles/:id
Vehicle	src/controllers/vehicleController.js	createVehicle	130 – 209	POST /api/vehicles
Vehicle	src/controllers/vehicleController.js	updateVehicle	216 – 281	PUT /api/vehicles/:id
Vehicle	src/controllers/vehicleController.js	deleteVehicle	288 – 309	DELETE /api/vehicles/:id
Driver	src/controllers/driverController.js	getDrivers	26 – 62	GET /api/drivers
Driver	src/controllers/driverController.js	getDriverById	69 – 103	GET /api/drivers/:id
Driver	src/controllers/driverController.js	getMyDriverProfile	110 – 133	GET /api/drivers/me
Driver	src/controllers/driverController.js	createDriver	140 – 240	POST /api/drivers
Driver	src/controllers/driverController.js	updateDriver	247 – 320	PUT /api/drivers/:id
Driver	src/controllers/driverController.js	assignVehicle	327 – 388	PUT /api/drivers/:id/assign
Driver	src/controllers/driverController.js	deleteDriver	395 – 416	DELETE /api/drivers/:id
Trip	src/controllers/tripController.js	getTrips	26 – 95	GET /api/trips
Trip	src/controllers/tripController.js	getTripById	102 – 126	GET /api/trips/:id
Trip	src/controllers/tripController.js	createTrip	133 – 206	POST /api/trips
Trip	src/controllers/tripController.js	updateTripStatus	213 – 260	PATCH /api/trips/:id/status
Trip	src/controllers/tripController.js	updateTrip	267 – 325	PUT /api/trips/:id
Trip	src/controllers/tripController.js	deleteTrip	332 – 348	DELETE /api/trips/:id

Model / Collection	Controller File Path	Controller Function	Source Lines	API Endpoint / Method
Trip	src/controllers/tripController.js	getTripRoute	563 – 590	GET /api/trips/:id/route
FuelEntry	src/controllers/fuelController.js	getFuelEntries	25 – 86	GET /api/fuel
FuelEntry	src/controllers/fuelController.js	createFuelEntry	93 – 192	POST /api/fuel
FuelEntry	src/controllers/fuelController.js	getWeeklySummary	199 – 235	GET /api/fuel/weekly-summary
FuelEntry	src/controllers/fuelController.js	deleteFuelEntry	242 – 257	DELETE /api/fuel/:id
MaintenanceRecord	src/controllers/maintenanceController.js	getMaintenanceRecords	25 – 71	GET /api/maintenance
MaintenanceRecord	src/controllers/maintenanceController.js	createMaintenanceRecord	78 – 151	POST /api/maintenance
MaintenanceRecord	src/controllers/maintenanceController.js	updateMaintenanceStatus	158 – 203	PATCH /api/maintenance/:id/status
MaintenanceRecord	src/controllers/maintenanceController.js	deleteMaintenanceRecord	210 – 226	DELETE /api/maintenance/:id
Activity	src/controllers/dashboardController.js	getActivities	165 – 192	GET /api/dashboard/activities
Activity	src/controllers/dashboardController.js	getDashboardStats	15 – 158	GET /api/dashboard/stats